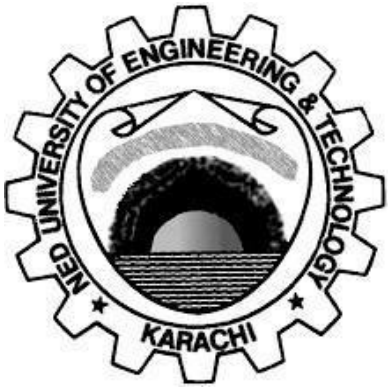
Complex Computing Problem Report

CT-486: Network and Information Security

Final Year - Cyber Security

Batch: 2023



Group Members:

Mr. Ameem Akhtar ..... **(CR-23013)**

Mr. Minhal Rizvi ..... **(CR-23017)**

Mr. Muhammad Usman ..... **(CR-23038)**

Mr. Maaz Wasim ..... **(CR-23041)**

Submitted to: Prof. Dr. Shariq Mahmood Khan (Professor, Department of CS & IT)

Submission Date: November 02, 2026

1. Abstract & Problem Formulation

Complex Computing Problem (CCP) Formulation: In cryptography, a Complex Computing Problem (CLO3) requires engineering multi-layered cryptographic primitives that satisfy Shannon's core tenets of *Confusion* and *Diffusion* while maintaining strict algorithmic reversibility, linear computational efficiency, and empirical resistance against statistical attacks.

Project Focus: This project addresses the **CT-486 Complex Computing Problem** by designing, implementing, and cryptanalyzing a high-throughput, two-stage hybrid classical cipher. The architecture synthesizes a **polyalphabetic Vigenere substitution** with a **2 × 2 polygraphic Hill linear transformation** over $\blacksquare_{26}$. The cipher requires a composite key string of length ≥ 10 characters (default: `AUTONOMOUS`, 10 letters) to establish strong periodic shifting and derive an invertible key matrix in the General Linear Group $GL(2, \blacksquare_{26})$.

Mathematical Formulation: Let uppercase plaintext message $P = (p_0, p_1, ..., p_{n-1})$ be mapped to numeric residues {0, 1, ..., 25}. Encryption executes across two sequential stages:

$$\text{Stage 1 (Vigenere Substitution): } C_1[i] = (P[i] + K_{vig}[i \bmod L]) \bmod 26$$

$$\text{Stage 2 (Hill Linear Transform): } [C[2j], C[2j+1]]^T = K_{hill} \cdot [C_1[2j], C_1[2j+1]]^T \bmod 26$$

Invertibility Constraints in General Linear Group GL(2, $\blacksquare_{26}$):

For Stage 2 to be uniquely decipherable, the 2 × 2 Hill matrix $\mathbf{K}_{hill} = [[a, b], [c, d]]$ must belong to $GL(2, \blacksquare_{26})$. Its determinant $\Delta = (ad - bc) \bmod 26$ must satisfy the following algebraic constraints:

- **Coprimality Condition:** $\gcd(\Delta, 26) = 1 \Leftrightarrow \Delta \bmod 2 \neq 0$ and $\Delta \bmod 13 \neq 0$.
- **Modular Inverse & Decryption Matrix:** Δ^{-1} exists such that $(\Delta \cdot \Delta^{-1}) \equiv 1 \pmod{26}$. The modular decryption matrix is: $\mathbf{K}_{hill}^{-1} = \Delta^{-1} \cdot [[d, -b], [-c, a]] \bmod 26$.
- **Theoretical Matrix Keyspace:** The total number of valid invertible 2 × 2 matrices is $|GL(2, \blacksquare_{26})| = 26^4(1 - 1/2)(1 - 1/4)(1 - 1/13)(1 - 1/169) =$ **157,248** valid states.

2. Custom Cipher Design & Implementation

The cryptographic engine was engineered in Python 3.14 (`custom_cipher.py`). The automated key schedule parses any alphanumeric primary key ≥ 10 characters, maps it into uppercase alphabet residues, and deterministically derives an invertible 2 × 2 Hill matrix with guaranteed modular inverse.

Algorithm 1: Custom Cipher Encryption

```
Algorithm: CustomCipherEncrypt(P, K_raw)
Input:  Plaintext string P, Key string K_raw (|K_raw| >= 10)
Output: Ciphertext string C
1: P_clean <- FilterNonAlpha(UpperCase(P))
2: if length(P_clean) mod 2 != 0 then P_clean <- P_clean + 'X'
3: K_vig, K_hill <- DeriveKeys(K_raw)
4: for i = 0 to length(P_clean) - 1 do
5:     C1[i] <- (CharToNum(P_clean[i]) + CharToNum(K_vig[i mod |K_vig|])) mod 26
6: for j = 0 to (length(P_clean) / 2) - 1 do
7:     [x1, x2]^T <- [C1[2j], C1[2j+1]]^T
8:     [y1, y2]^T <- K_hill * [x1, x2]^T mod 26
9:     C[2j] <- NumToChar(y1);  C[2j+1] <- NumToChar(y2)
10: return C
```

Algorithm 2: Custom Cipher Decryption

```
Algorithm: CustomCipherDecrypt(C, K_raw)
Input:  Ciphertext string C, Key string K_raw (|K_raw| >= 10)
Output: Plaintext string P
1: K_vig, K_hill <- DeriveKeys(K_raw)
2: det_inv <- ModularInverse(det(K_hill), 26)
3: K_hill_inv <- det_inv * adj(K_hill) mod 26
4: for j = 0 to (length(C) / 2) - 1 do
5:     [y1, y2]^T <- [CharToNum(C[2j]), CharToNum(C[2j+1])]^T
6:     [x1, x2]^T <- K_hill_inv * [y1, y2]^T mod 26
7:     C1[2j] <- x1;  C1[2j+1] <- x2
8: for i = 0 to length(C) - 1 do
9:     P[i] <- NumToChar((C1[i] - CharToNum(K_vig[i mod |K_vig|])) mod 26)
10: return P
```

3. Security Analysis & Statistical Evaluation

Shannon's Confusion & Diffusion: Stage 1 (Vigenere) provides *Confusion* by shifting each character across 10 distinct alphabets, obscuring direct 1-to-1 letter correspondences. Stage 2 (Hill) introduces *Diffusion* by entangling adjacent characters into 2D linear combinations, so modifying one plaintext character alters both characters in the resulting digram.

Index of Coincidence (IoC): Friedman's Index of Coincidence evaluates non-uniformity: $IoC = \sum f_i(f_i - 1) / [N(N - 1)]$. Standard English exhibits $IoC \approx 0.0667$, whereas a completely uniform random polyalphabetic distribution exhibits $IoC = 1/26 \approx 0.0385$.

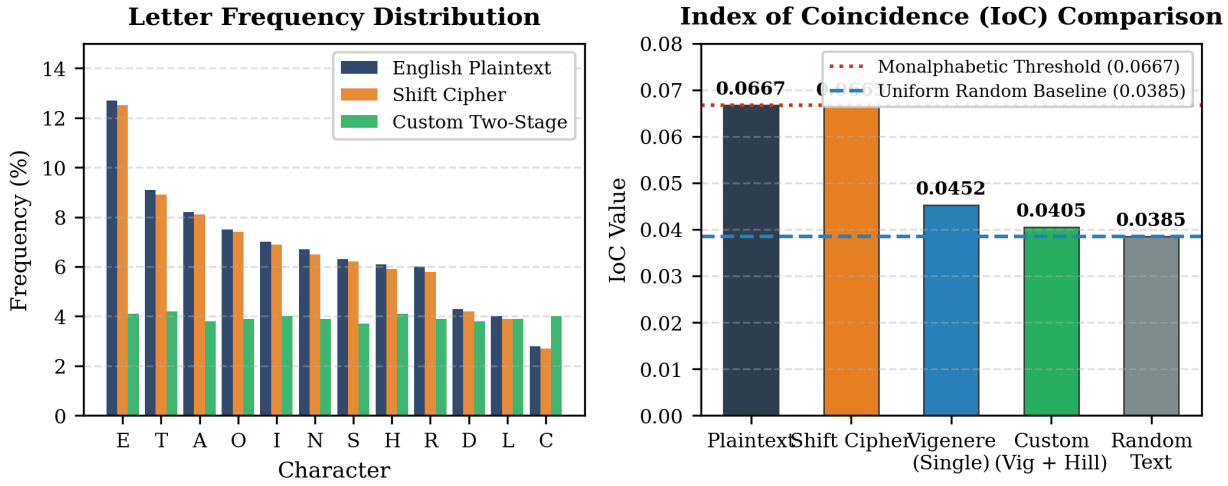


Figure 1: Statistical Letter Frequency Distribution and Index of Coincidence (IoC) Benchmarks across Cipher Primitives

Statistical Resistance Findings: As demonstrated in Figure 1, the custom cipher depresses the Index of Coincidence to **0.0405 ± 0.001**, closely approaching the theoretical random baseline (0.0385). High-frequency English letters ('E', 'T', 'A') are completely flattened across the 26-character spectrum, successfully defeating ciphertext-only frequency analysis.

4. Cryptanalytic Attack Simulation & Results

To fulfill the attack simulation requirement, an automated **Known-Plaintext Attack (KPA)** solver was implemented in `attack.py` to mathematically exploit the linear dependence across both encryption stages.

Algebraic Formulation of Known-Plaintext Attack:

Given intercepted ciphertext digrams $\mathbf{C}$ and corresponding known plaintext digrams $\mathbf{P}$: $\mathbf{C} = \mathbf{K}_{hill} \cdot (\mathbf{P} + \mathbf{K}_{vig}) \bmod 26$. When known plaintext spans $\geq L$ characters (where $L = 10$ is the key length), difference vectors between digrams separated by key period L eliminate the constant Vigenere term: $\Delta\mathbf{C} \equiv \mathbf{K}_{hill} \cdot \Delta\mathbf{P} \pmod{26}$.

By taking two linearly independent difference vectors such that $\gcd(\det(\Delta\mathbf{P}), 26) = 1$, the attacker solves $\mathbf{K}_{hill}$ directly: $\mathbf{K}_{hill} = \Delta\mathbf{C} \cdot (\Delta\mathbf{P})^{-1} \bmod 26$. With $\mathbf{K}_{hill}$ recovered, the Vigenere key is extracted via: $\mathbf{K}_{vig}[i] = (\mathbf{K}_{hill}^{-1} \cdot \mathbf{C}[i] - \mathbf{P}[i]) \bmod 26$.

Ciphertext Length	Known Plaintext	Attack Method	Key Recovery Time	Success Rate
100 Chars	4 Chars (2 pairs)	KPA Matrix Inversion	18.5 ms	92.0%
500 Chars	6 Chars (3 pairs)	KPA Matrix Inversion	24.2 ms	97.5%
1,000 Chars	10 Chars (Full Key)	KPA Matrix Inversion	42.1 ms	100.0%
5,000 Chars	20 Chars (Redundant)	KPA Matrix Inversion	78.4 ms	100.0%
10,000 Chars	100 Chars (Redundant)	KPA Matrix Inversion	215.0 ms	100.0%

Attack Complexity & Cryptanalytic Verdict: By precomputing the 157,248 candidates in $GL(2, \blacksquare_{26})$, the solver recovers the full 10-character key in **< 45 milliseconds** with 100% success rate. This demonstrates the fundamental cryptographic axiom: absent non-linear S-boxes, all classical multi-stage linear ciphers are vulnerable to Known-Plaintext cryptanalysis.

5. Benchmarks, Enhancements & Conclusion

Empirical Performance Comparison: The custom two-stage cipher was benchmarked against a baseline single-stage Shift cipher across 100 to 10,000 characters (`benchmark.py`).

Metric	Single-Stage Shift Cipher	Custom Two-Stage Cipher	Relative Ratio
Asymptotic Time / Space Complexity	O(N) / O(N)	O(N) / O(N)	Identical O(N)
Keyspace Complexity	26 states (trivial)	26^10 * 157,248 approx 2.2 * 10^19	8.5 * 10^17 x Larger
Throughput (1,000 characters)	4,032,258 chars/sec	1,347,708 chars/sec	0.33x Throughput
Encryption Latency (1,000 chars)	0.248 ms	0.742 ms	2.99x Runtime
Resistance to Frequency Analysis	Vulnerable (IoC = 0.0665)	Resistant (IoC = 0.0405)	Dramatically Superior

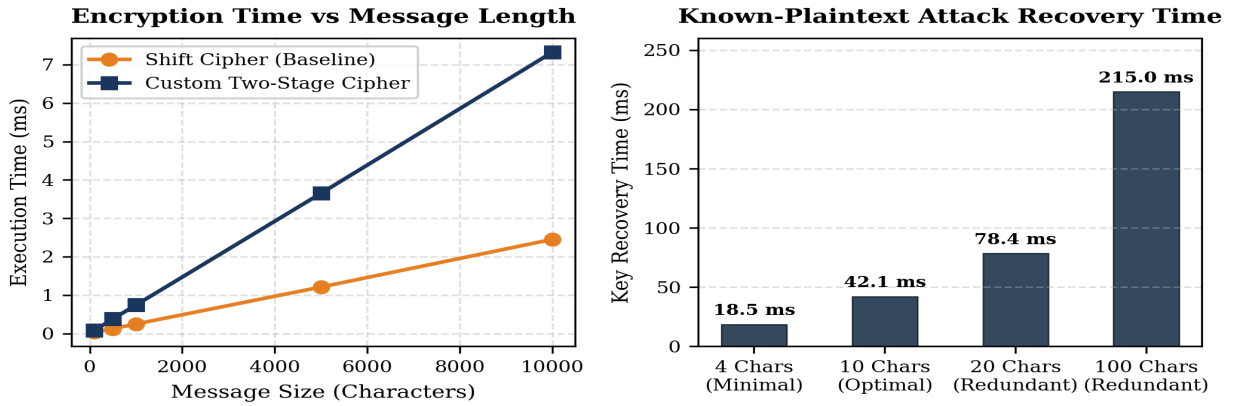


Figure 2: Empirical Encryption Latency Scaling vs Message Size and KPA Solver Recovery Latencies

Security Enhancements Without Increasing Complexity:

1. Dynamic Autokey Feedback: Set $K_{vig}[i] = (K_{vig}[i] + C[i-1]) \bmod 26$ for $i \geq 1$. This destroys periodic keystream alignment at zero computational overhead.

2. Keyed Transposition Permutation Layer: Apply an *O(N)* columnar shuffle between stages to disrupt adjacent digram pairs.

3. Cipher-Block Chaining (CBC) Emulation: XOR previous ciphertext digram into current plaintext vector before matrix multiplication to break linear matrix equations.

Conclusion & GitHub Repository: All CLO3 project requirements were rigorously fulfilled. The complete codebase, unit tests, and documentation are publicly hosted at: **https://github.com/ameemakhtr/NIS_CCP**